

Báo cáo rà soát test code

<!-- framework: Jest | run_id: run_20260617_000009_470539 -->

Framework: Jest | Run ID: run_20260617_000009_470539

Tóm tắt kỹ thuật

- Framework: Jest
- Kết quả chạy: Không có bằng chứng thực thi; các test chỉ là test.todo nên luôn được báo là “todo” và không thực thi.
- Coverage / Quality gate: Không có coverage vì không có test thực thi.
- Loại vấn đề thực thi: Import sai (không tồn tại ./source_under_test) và thiếu thực thi thực tế.
- Trình tự blocker quan trọng nhất: Các test chỉ là test.todo → không kiểm tra gì, không đáp ứng yêu cầu “Hybrid” và không cung cấp bất kỳ xác nhận nào.

Tài liệu kiểm thử được xem xét: Test plan JSON (TC-001 ... TC-006).

Mã nguồn được cung cấp: Không có mã nguồn thực tế; chỉ có một dòng `require('./source_under_test')` không tồn tại.

Lỗi nghiêm trọng

- Import không tồn tại
- Bằng chứng: `const target = require('./source_under_test');` trong mã kiểm thử, trong khi phần [SOURCE CODE] không cung cấp file `source_under_test.js`.
- Ảnh hưởng: Khi chạy Jest, Node sẽ ném lỗi `Cannot find module './source_under_test'`, khiến toàn bộ suite không thể khởi động.
- Cách sửa: Xóa dòng import nếu không cần, hoặc thay thế bằng đường dẫn tới module thực tế cần kiểm thử (ví dụ `require('pytest-cov')` nếu có wrapper JS, hoặc tạo một mock stub).

- Tất cả các test chỉ là test.todo
- Bằng chứng: Các test được khai báo như `test.todo('TC-001: ...');` ... `test.todo('TC-006: ...');`.
- Ảnh hưởng: test.todo không thực thi bất kỳ mã nào, không có assertion, không kiểm tra yêu cầu, do đó không đáp ứng mục tiêu “Hybrid” và không cung cấp kết quả kiểm thử.
- Cách sửa: Thay test.todo bằng `test('TC-001 ...', async () => { /* thực hiện yêu cầu, assert */ })` và viết các bước thực thi, dữ liệu đầu vào, và các assertion dựa trên yêu cầu.

Test còn thiếu

- Không có test thực tế cho bất kỳ yêu cầu nào (cài đặt, cú pháp, định dạng file, hạn chế, bảo mật, môi trường).
- Thiếu test âm tính / biên: ví dụ, kiểm tra lỗi khi cài đặt thất bại, hoặc khi tùy chọn CLI không hợp lệ.
- Thiếu test bảo mật: không có kiểm tra việc xử lý dữ liệu coverage trong CI, không có mô phỏng tấn công injection.
- Thiếu test tích hợp: không có mô phỏng chạy `pytest --cov` và kiểm tra file `.coverage` được tạo.

Assertion yếu

- Không có assertion nào vì mọi test đều là test.todo.
- Không có kiểm tra giá trị trả về, không có kiểm tra nội dung tài liệu, không có kiểm tra lỗi.

Rủi ro maintainability

- Phụ thuộc vào module không tồn tại (./source_under_test) → khi thêm file thực tế, có thể gây xung đột nếu không cập nhật import.
- Todo tests thường bị bỏ quên; nếu không chuyển thành test thực, bộ kiểm thử sẽ luôn “green” mà không kiểm chứng gì, gây hiểu lầm về chất lượng.
- Không có mock hoặc stub → khi viết test thực tế, cần xác định rõ phụ thuộc (ví dụ, gọi CLI, đọc file) để tránh phụ thuộc vào môi trường thực.

Gợi ý sửa ngay

1. Xóa hoặc sửa import

// Nếu không cần module nào:

// const target = require('./source_under_test'); // Xóa dòng này

// Hoặc thay bằng module thực tế:

const pytestCov = require('pytest-cov'); // ví dụ, nếu có wrapper JS

2. Chuyển test.todo thành test thực

test('TC-001: Clarify installation steps', async () => {

// Giả sử có hàm getInstallationInfo() trả về chuỗi

const info = await getInstallationInfo();

expect(info).toMatch(/pip install pytest-cov/);

});

3. Thêm ít nhất một assertion cho mỗi TC dựa trên yêu cầu (kiểm tra chuỗi, bảng, JSON, v.v.).

4. Thêm test âm tính / biên cho các trường hợp lỗi (cú pháp CLI sai, file .coverage hỏng).

5. Mock các phụ thuộc bên ngoài (ví dụ, child_process.exec để chạy lệnh pytest --cov) để tránh chạy thực tế trong CI.

6. Cập nhật Test Plan: Đánh dấu các TC đã chuyển thành test thực và bổ sung các TC mới cho các kịch bản âm tính, bảo mật, và tích hợp.